

Mathematical Example of a Convolutional Neural Network (CNN)

Objective: Demonstrate how a CNN processes a simple 2D input using a convolution layer, ReLU activation, max pooling, and a dense layer.

1. Input Image (4x4 Grayscale Matrix)

$$\text{Input} = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 3 & 1 & 1 & 2 \\ 1 & 0 & 2 & 3 \\ 2 & 1 & 2 & 1 \end{bmatrix}$$

2. Convolution Filter (2x2)

$$\text{Filter} = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

Stride = 1, Padding = 0 (valid)

3. Convolution Output (Before ReLU)

We slide the 2x2 filter over the image. First region:

$$\begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix} \Rightarrow 1 \times 1 + 2 \times 0 + 3 \times (-1) + 1 \times 1 = 1 - 3 + 1 = -1$$

Repeat this over the image to get:

$$\text{Feature Map (before ReLU)} = \begin{bmatrix} -1 & 2 & 0 \\ 2 & 1 & 2 \\ 0 & 3 & 2 \end{bmatrix}$$

4. Apply ReLU Activation

Replace negative values with 0:

$$\text{Feature Map (after ReLU)} = \begin{bmatrix} 0 & 2 & 0 \\ 2 & 1 & 2 \\ 0 & 3 & 2 \end{bmatrix}$$

5. Max Pooling (2x2, Stride=2)

Divide into 2x2 blocks and take maximum:

$$\text{Block 1: } \begin{bmatrix} 0 & 2 \\ 2 & 1 \end{bmatrix} \Rightarrow \max = 2$$

$$\text{Block 2: } \begin{bmatrix} 1 & 2 \\ 3 & 2 \end{bmatrix} \Rightarrow \max = 3$$

Result:

$$\text{Pooled Output} = [2, 3]$$

6. Fully Connected Layer (Dense Layer)

Assume weight matrix W and bias b :

$$W = \begin{bmatrix} 0.5 & -1.0 & 0.3 \\ 0.2 & 0.4 & 0.1 \end{bmatrix}, \quad b = [0.1, 0.2, -0.3]$$

Input vector = $[2, 3]$

Dot product:

$$[2, 3] \cdot W = [2 \times 0.5 + 3 \times 0.2, 2 \times (-1.0) + 3 \times 0.4, 2 \times 0.3 + 3 \times 0.1] = [1.6, -0.8, 0.9]$$

Add bias:

$$[1.6 + 0.1, -0.8 + 0.2, 0.9 - 0.3] = [1.7, -0.6, 0.6]$$

7. Softmax Output

Apply softmax to get class probabilities:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Compute exponentials:

$$e^{1.7} \approx 5.47, \quad e^{-0.6} \approx 0.55, \quad e^{0.6} \approx 1.82$$

$$\text{Sum} = 5.47 + 0.55 + 1.82 = 7.84$$

$$\text{Softmax Output} = \left[\frac{5.47}{7.84}, \frac{0.55}{7.84}, \frac{1.82}{7.84} \right] \approx [0.698, 0.070, 0.232]$$

 Final Result:

The model predicts **Class 0** with probability ~**69.8%**

CNN Mathematical Practice Example

Here's a new unsolved CNN mathematical example you can try solving on your own. It follows the same

flow: input → convolution → ReLU → pooling → dense → softmax.

1. Input Image (4×4 Grayscale Matrix)

$$\text{Input} = \begin{bmatrix} 2 & 1 & 0 & 2 \\ 0 & 1 & 3 & 1 \\ 1 & 2 & 1 & 0 \\ 2 & 1 & 0 & 2 \end{bmatrix}$$

2. Convolution Filter (2×2 Kernel)

$$\text{Filter} = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$$

- Stride: 1
- Padding: 0 (valid)

3. Activation Function

Use **ReLU** after convolution.

4. Max Pooling Layer

- Pool size: 2×2
- Stride: 2

5. Fully Connected Layer (Dense)

- Flatten the pooled output
- Use the following weight matrix W and bias vector b :

$$W = \begin{bmatrix} 0.3 & -0.6 & 0.2 \\ 0.4 & 0.5 & -0.1 \end{bmatrix}, \quad b = [0.2, -0.3, 0.1]$$

6. Output Layer

- Apply **softmax** to the final scores to get class probabilities.